© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_8

8. Spring AI Ninja Moves

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to build an LLM-powered application using the Spring AI ecosystem of Spring Boot.

Introduction

So far we have explored the LangChain4j package and the various features and utilities it provides. Although it's useful and is modularized, we always want to be familiar with other open-source community developer libraries that could also help us with building applications. This is how application developers can build a healthy ecosystem. If you have built Spring Boot applications before using the handy packages of Spring like Spring Web and Spring Data JPA, you know how easy and seamless the developer experience is. Fortunately, there is something similar for the LLM ecosystem too. The Spring AI ecosystem can help with different integrations similar to the ones we built with LangChain4j. In the following sections, you will learn more about Spring AI, including the features and integrations it brings to the ecosystem, and build an application using it.

What Is Spring AI?

Spring AI is an application framework for AI engineering. Its goal is to apply Spring ecosystem design principles such as portability and modular design to the AI domain and promote using Plain Old Java Objects (POJOs) as the building blocks of an application. Figure 8-1 shows the interaction model between Spring Boot and the Gen AI interface through the Spring AI ecosystem.

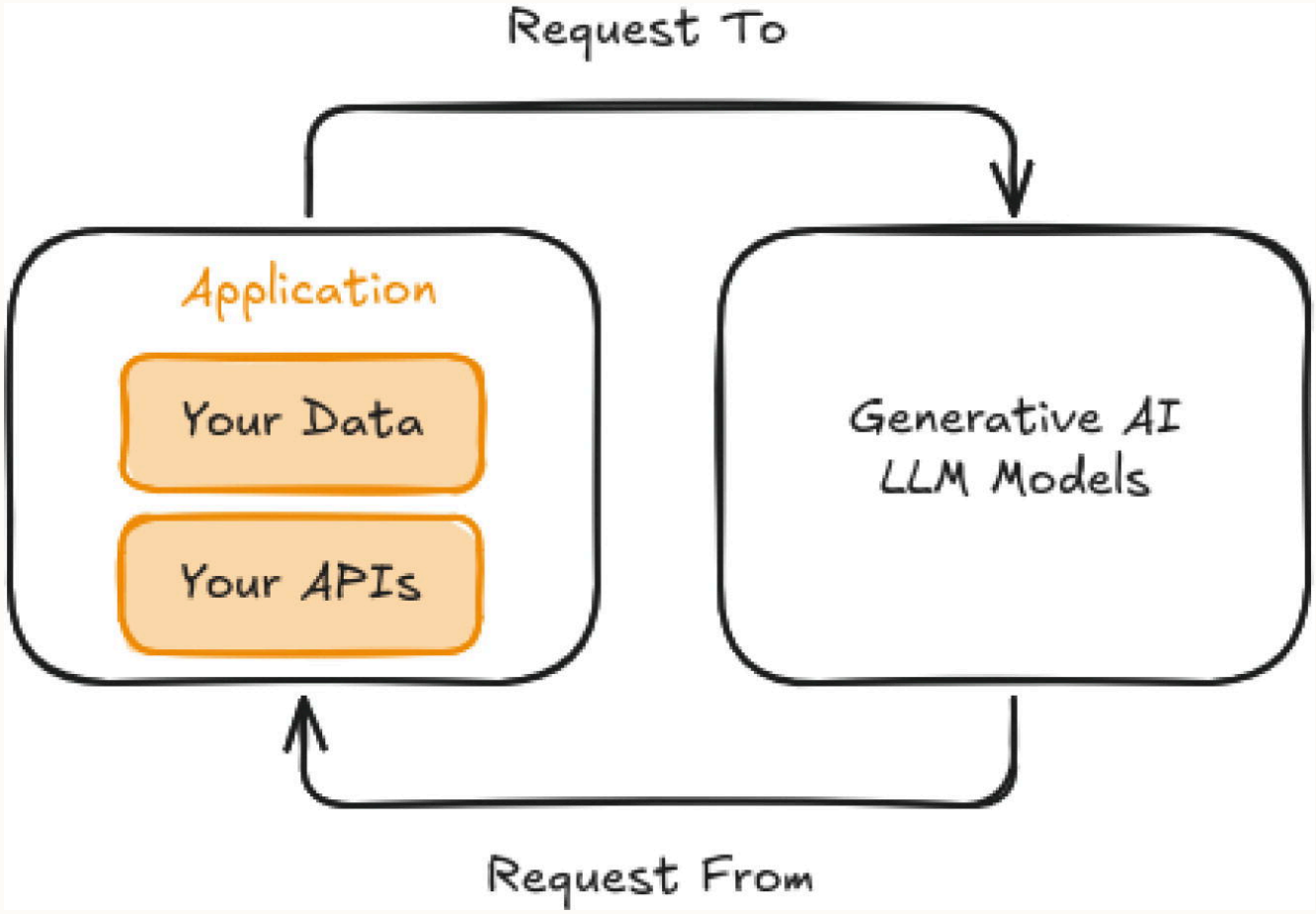


Figure 8-1 Request-response flow between Spring Boot (Spring AI) and Gen AI Interface

At its core, Spring AI addresses the fundamental challenge of AI integration: connecting your enterprise data and APIs with the AI models.

Features of Spring AI

Spring AI offers extensive support for major AI model providers, including Anthropic, OpenAI, Microsoft, Amazon, Google, and Ollama. It accommodates various model types such as chat completion, embeddings, text-to-image, audio transcription, text-to-speech, and moderation. The framework provides a portable, unified API that works seamlessly across different AI providers, supporting both synchronous and streaming interactions and allowing access to provider-specific features. Additionally, Spring AI simplifies integration through structured outputs, mapping model responses directly to POJOs.

In terms of vector databases, Spring AI supports prominent providers like Apache Cassandra, Azure Vector Search, Chroma, Milvus, MongoDB Atlas, Neo4j, Oracle, PostgreSQL/PGVector, Pinecone, Qdrant, Redis, and Weaviate. It includes a portable API that spans these vector store providers, featuring an innovative SQL-like metadata filtering capability.

The toolkit also includes advanced capabilities such as tools/function calling, enabling AI models to invoke client-side tools or functions for real-time information retrieval. It promotes observability by providing detailed insights into AI operations. A built-in document injection ETL framework facilitates data engineering processes, while AI model evaluation utilities help ensure generated content's reliability and prevent hallucinated responses. For a complete discussion of choosing metrics, traces, and logs, please refer to Chapter 18.

For developers, Spring AI delivers the ChatClient API, a fluent, intuitive interface similar to WebClient and RestClient APIs, simplifying communication with AI chat models. The Advisors API further encapsulates common generative AI patterns, manages data transformation, and enhances portability across various models and use cases. Additionally, Spring AI supports conversation memory and retrieval-augmented generation (RAG) capabilities.

Spring Boot Auto Configuration and starters are available for effortless setup and integration of AI models and vector stores, accessible via the Spring Initializr (start.spring.io).

Boot Up an Application Using Spring AI

Create a Spring Boot web application with a Spring AI OpenAI boot starter dependency. This Spring Initializr link can help you bootstrap the application:

https://start.spring.io/#!type=maven-project&language=java&platformVersion=3.3.4&packaging=jar&jvmVersion=17&groupId=spring.ai.example&artifactId=spring-ai-demo&name=spring-ai-demo&description=Spring%20AI%20%2C%20getting%20started%20example%2C%20using%20Open%20AI&packageName=spri

Note: With start.spring.io you can select any AI models or vector stores that you want to use in your new applications.

Start by typing spring ai in the search box, which will show you many packages supported through the Spring AI ecosystem. Figure 8-2 shows the list of packages/libraries which are supported at this moment. As in the Gen AI world, nothing is fixed, and this list may change.

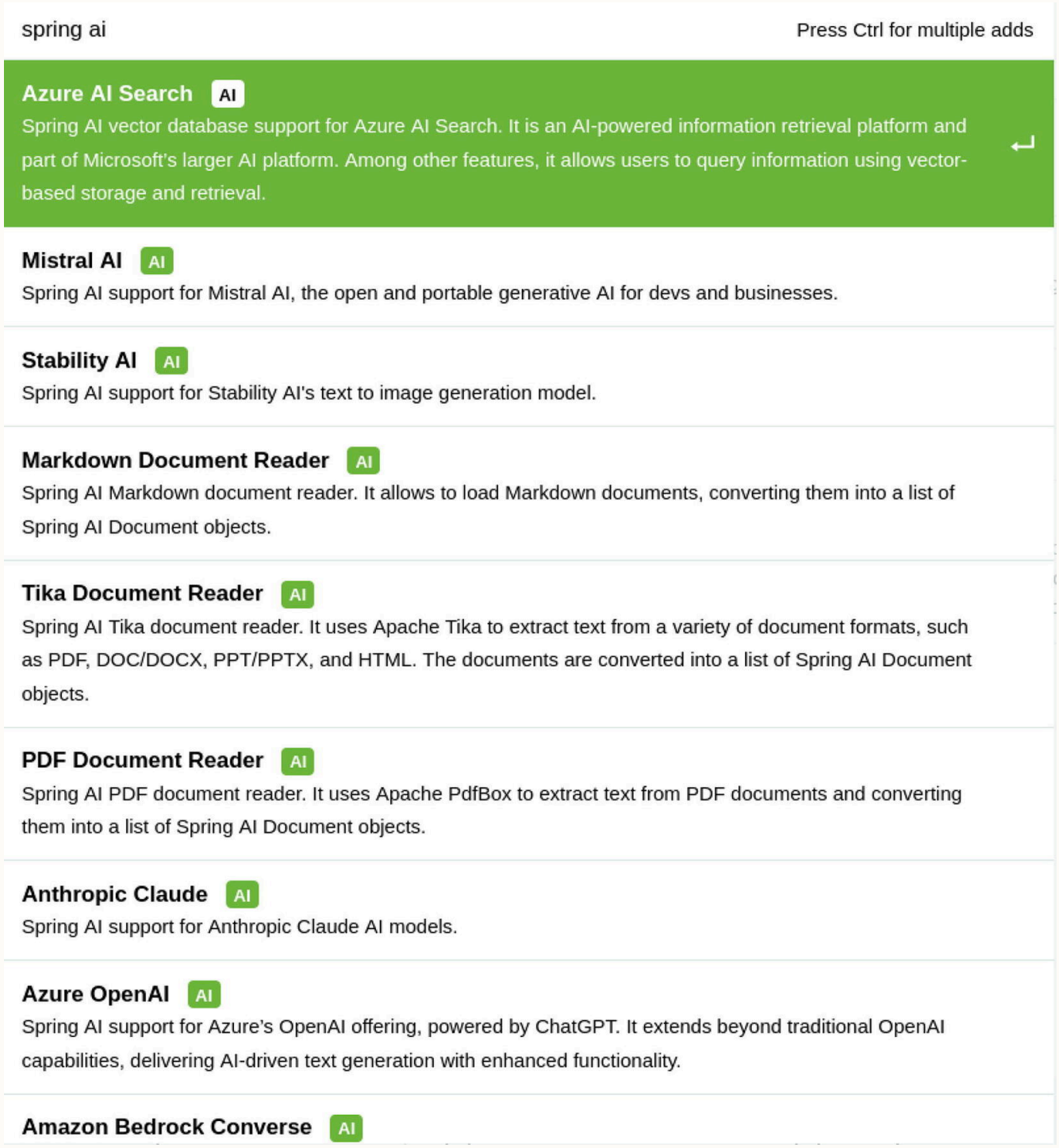


Figure 8-2 Spring AI-supported packages

You will start by selecting a few packages that will be essential to your project, as depicted in Figure 8-3.

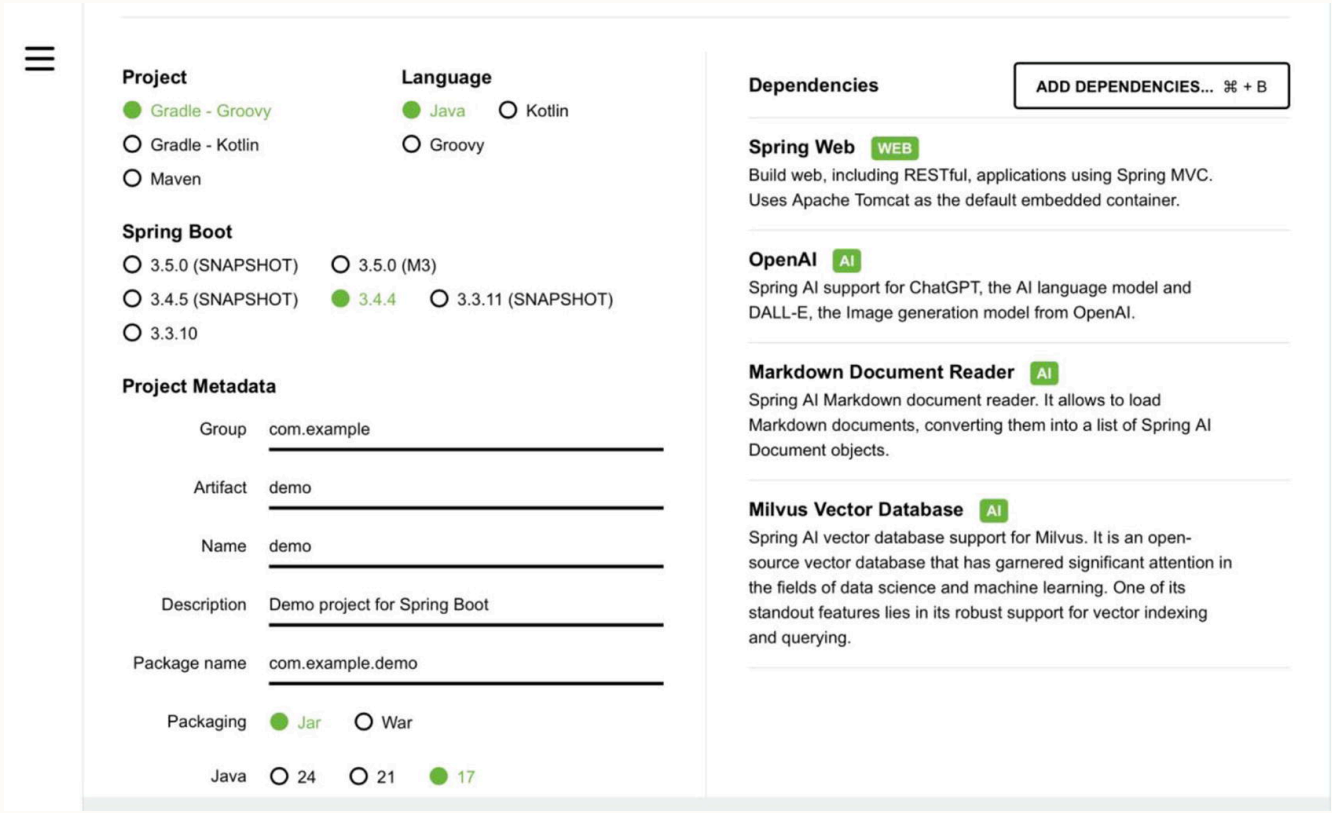


Figure 8-3 Spring Boot package dependencies in Spring Initializr UI

The following are the necessary packages:

- Spring Web: This is needed for building a REST endpoint for your chat interface.
- OpenAI: This package is necessary to connect to OpenAI and interact with their LLMs.
- Markdown Document Reader: This package helps to read contents of a Markdown file and generate document objects from it, which is a common format readable by all vector databases in the Spring AI ecosystem.
- Milvus Vector Database: As you have learned earlier, you need an embedded vector database to store your embeddings. Milvus is an open-source vector database, and this package will help to connect and interact with it.

Once the dependencies have been selected, download the archive. Unzip the demo zip archive that was downloaded to your system and open it in your IDE of choice. Let your IDE import the project-specific libraries.

For the next step, you will need an API key from OpenAI, which you can create at https://platform.openai.com/api-keys, as depicted in Figure 8-4. Please note that this is the web page at this moment in time. It could change in the future based on OpenAI's direction. Make sure to search in Google or the site map of OpenAI to create an API for your use case.

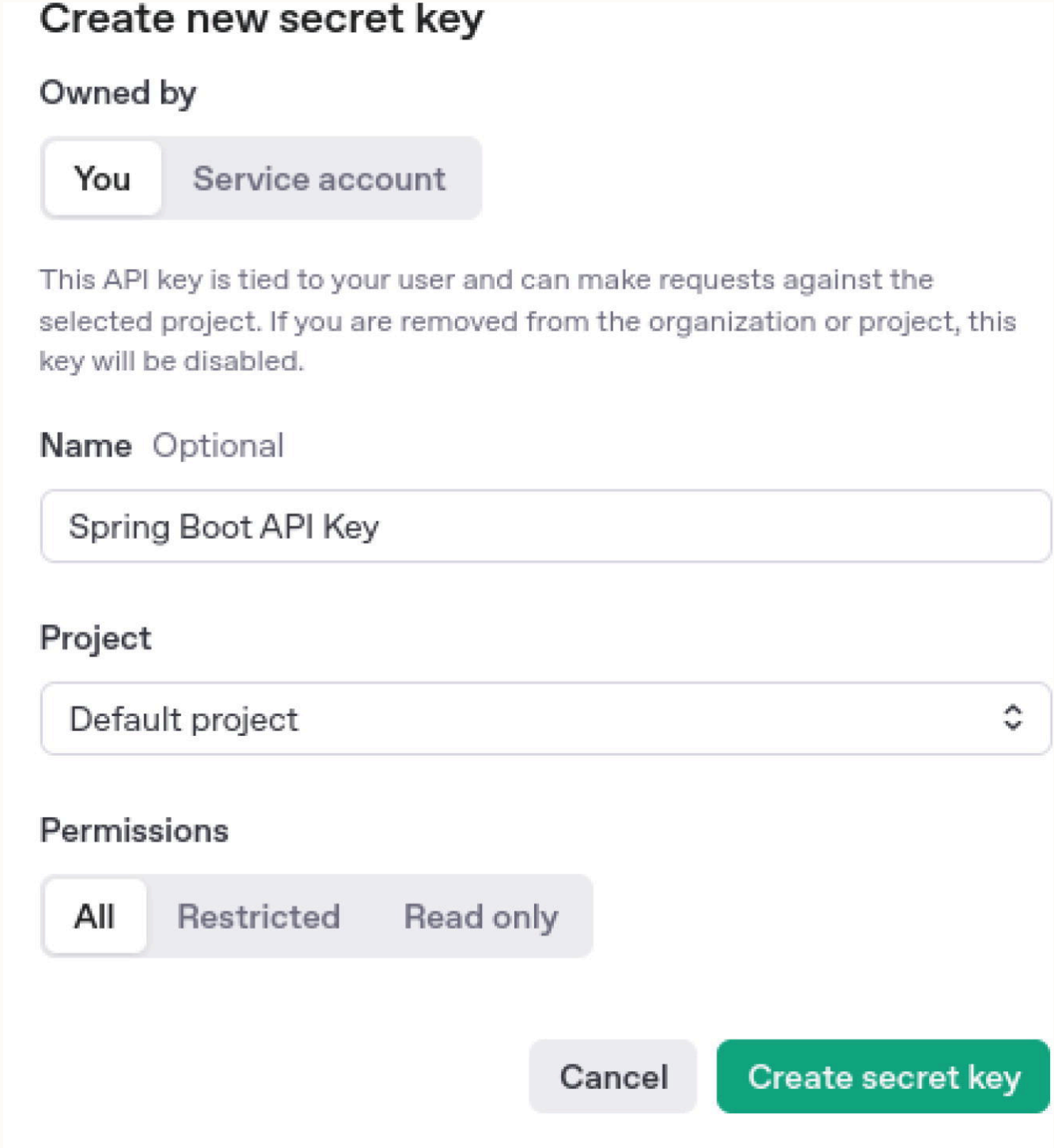


Figure 8-4 Create an API secret key token from the Open AI platform page

Once a secret key is created, copy it to safe place from the UI interface, as shown in Figure 8-5, since once this interface is closed, you won't have further access to it.

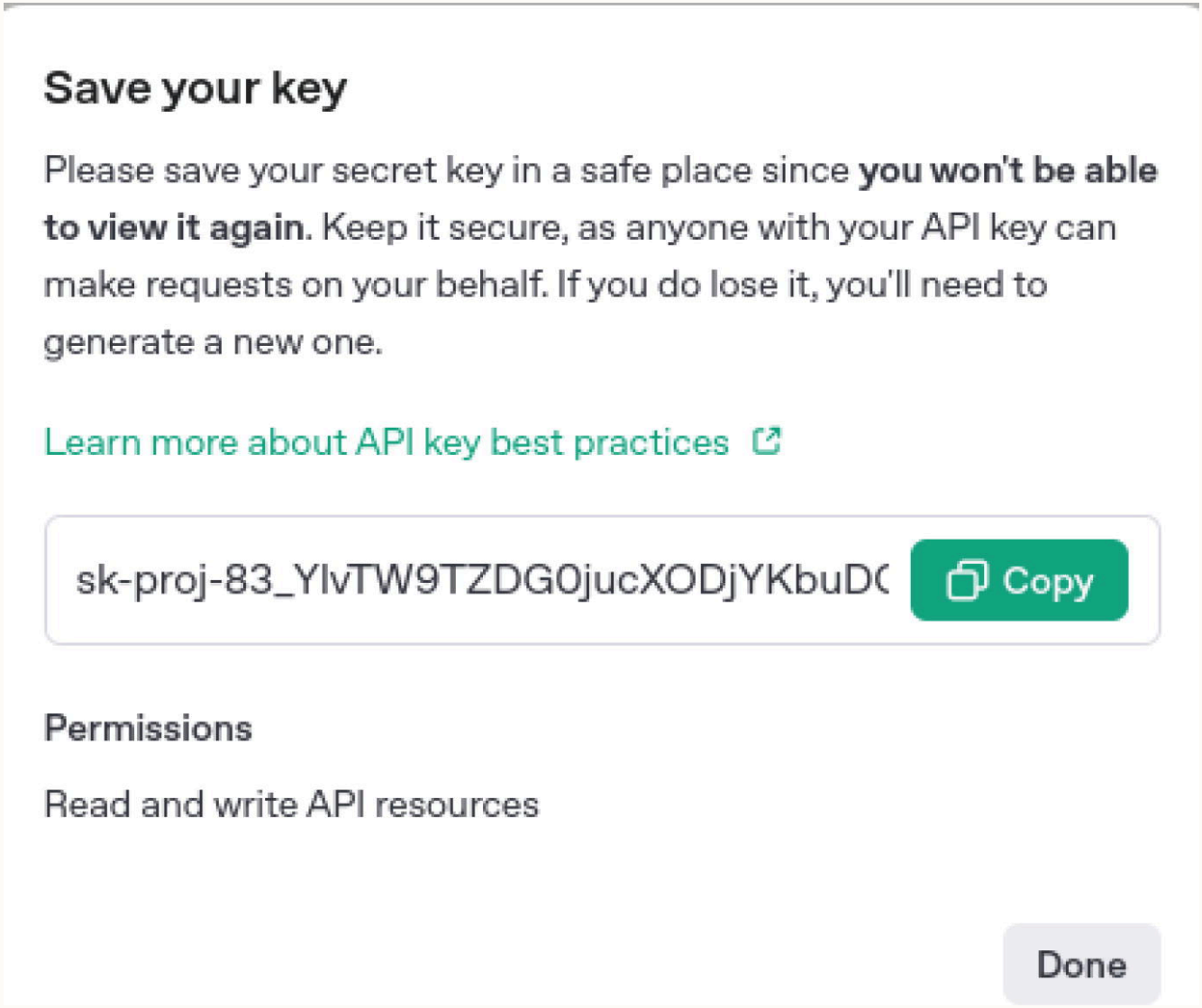


Figure 8-5 Copy the secret key in the Open AI interface

Add your OpenAI key to the application.properties file:

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=<YOUR OPENAI KEY>
```

At this point, you want to get something very quickly set up to get started talking to the OpenAI LLMs. For this example, you will use the ChatClient API. The ChatClient provides a fluent API for communicating with AI models. It supports both synchronous and streaming programming models.

Using ChatClient, you can build prompts, which as you know are essentially sets of messages guiding the AI's responses. These messages can be either user inputs or system-generated instructions, and they can include placeholders that dynamically adapt responses based on user-provided information. Additionally, the API also lets you configure specific options, like selecting the AI mode you prefer or adjusting the "temperature," which determines how creative or random the AI-generated responses will be.

ChatClient is created using the ChatClient.Builder object. You can use an autoconfigured ChatClient.Builder instance for any ChatModel Spring Boot autoconfiguration or create one programmatically. For this example, you will use an autoconfigured ChatClient.Builder. In the simplest case, Spring AI provides Spring Boot autoconfiguration, creating a prototype ChatClient.Builder bean for you to inject in your class.

You will create a bean with the CommandLineRunner type to enable running this method when Spring Boot is started and see the output at the command line. The builder instance is injected for you by Spring Boot, and you use this to build an instance of ChatClient. Using the chatClient object, you then create a prompt to tell a joke, call the request, and collect the response in the response variable. The response is then printed out.

Add the following snippet to your SpringAiDemoApplication class:

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
  public static void main(String[] args) {
     SpringApplication.run(DemoApplication.class, args);
   }
  @Bean
   public CommandLineRunner runner(ChatClient.Builder builder) {
     return args -> {
        ChatClient chatClient = builder.build();
        String response = chatClient.prompt("Tell me a joke").call().content();
        System.out.println(response);
     };
  }
}
```

Run the application and you should see output similar to the following in the terminal run space:

```
com.example.demo.DemoApplication          : Started DemoApplication in 4.576 seconds (process running for 5.886)
Why did the scarecrow win an award?
Because he was outstanding in his field!
```

This example shows how easy, quick, and intuitive it is to get started building Gen AI applications using Spring Boot and Spring AI.

Using OpenAI API Through Ollama API in Spring AI

In this section, you will learn about using the familiar OpenAI APIs in Spring AI but without the need to create an account or a payment license for it. You will proxy all the OpenAI API-specific requests through the Ollama API for OpenAI (see Figure 8-6).

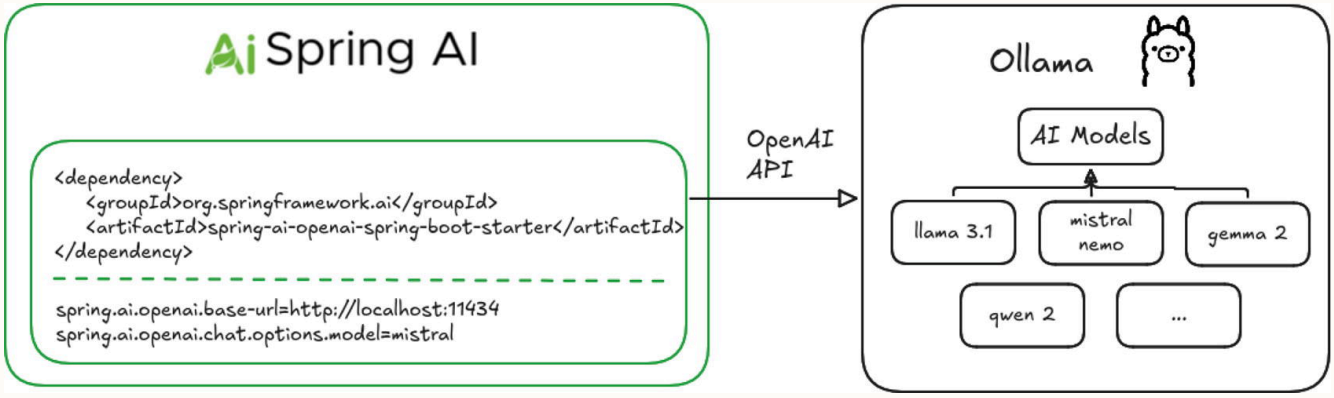


Figure 8-6 Spring AI Open AI to Ollama connection

To start, go to the Spring initializr website and choose the following dependencies: Spring Web, OpenAI, and Ollama, as shown in Figure 8-7.

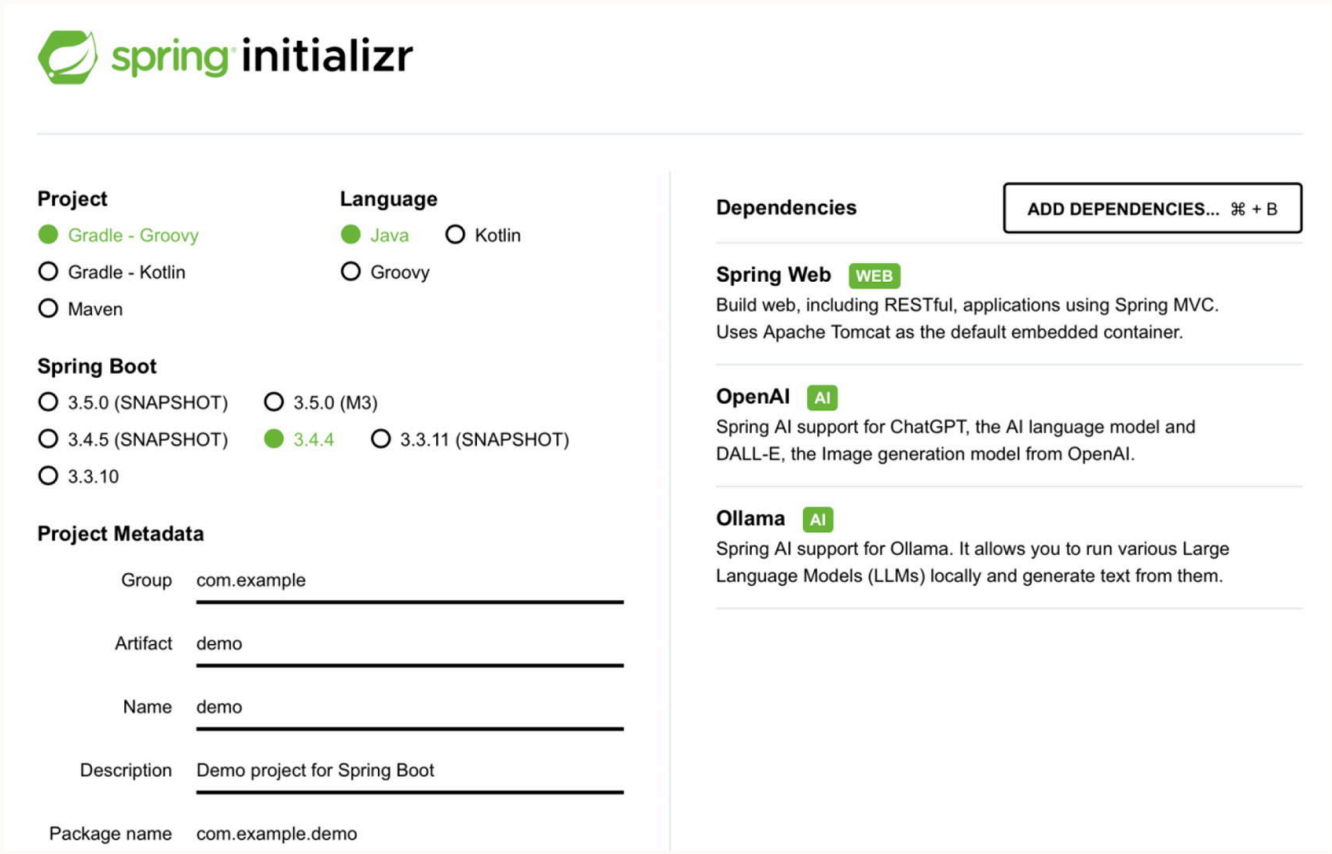


Figure 8-7 Dependency selection in Spring Initializr

Once the zip is downloaded, unzip it and open it in your favorite IDE to make sure everything works fine at this stage. Also as a prerequisite, make sure you have Ollama enabled and have the gemma3 model pulled on to your local machine using the following command. This is the 1 billion parameter version of the gemma3 model. You can also choose any other model of your choice as long as it fits the memory requirements of your system.

```
ollama pull gemma3:1b
```

Now, you will set a few config parameters in your application.properties file to enable OpenAI and Ollama to work. For Ollama, you need to let Spring AI know the base API URL for Ollama and which model and temperature of the creativity value you want to use for the LLM model.

```
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.temperature=0.7
spring.ai.ollama.chat.model=gemma3:1b
```

Once Ollama is set up for Spring AI, you will need to set the API key for OpenAI, which would be empty in this case; proxy the OpenAI API requests through the Ollama API URI; and set the model to be used with OpenAI in Spring AI.

```
spring.ai.openai.api-key=""
spring.ai.openai.chat.base-url=http://localhost:11434
spring.ai.openai.chat.options.model=gemma3:1b
```

The following are all the code changes for configuration in the application.properties file:

File: ./src/main/resources/application.properties

```
spring.application.name=demo
spring.ai.openai.api-key=""
spring.ai.openai.chat.base-url=http://localhost:11434
spring.ai.openai.chat.options.model=gemma3:1b
spring.ai.ollama.base-url=http://localhost:11434
spring.ai.ollama.chat.options.temperature=0.7
spring.ai.ollama.chat.model=gemma3:1b
```

Now that the connection from OpenAI to Ollama is established, let's try using the OpenAI chatModel. In the DemoApplication.java file, you will create a bean of the CommandLineRunner type, which takes in an injected argument of the OpenAiChatModel type. This will enable you to quickly prototype and test without establishing conventional controllers and then invoke it through a curl request.

```
@Bean
public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
   return args -> {
```

Next you will use the ChatClient API to create an instance and then customize it by passing a custom prompt and then setting the configuration parameters like model, temperature, and tokens settings.

```
ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
```

Then customize the prompt and set configuration parameters. Here you are setting a prompt for the LLM to classify movie reviews and then sharing an movie description to share the LLM's review.

```
String reviewSentiment = chatClient.prompt("""
                Classify movie reviews as POSITIVE,
                NEUTRAL or NEGATIVE. Review: "Her" is
                a disturbing study revealing the
                direction humanity is headed if AI is
                allowed to keep evolving, unchecked. I
                wish there were more movies like this
                masterpiece. Sentiment:
                """)
      .options(ChatOptions.builder()
            .model("gemma3:1b")
            .temperature(0.1)
            .maxTokens(5)
            .build());
```

The full code with all the changes is as follows:

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.prompt.ChatOptions;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
@SpringBootApplication
public class DemoApplication {
   enum Sentiment {
      POSITIVE, NEUTRAL, NEGATIVE
   }
  public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
  @Bean
   public CommandLineRunner commandLineRunner(OpenAiChatModel openAiChatModel) {
      return args -> {
         ChatClient chatClient = ChatClient.builder(openAiChatModel).build();
         String reviewSentiment = chatClient.prompt("""
                        Classify movie reviews as POSITIVE, NEUTRAL or
                        NEGATIVE. Review: "Her" is a disturbing study
                        revealing the direction humanity is headed if AI is
                        allowed to keep evolving, unchecked. I wish there were
                        more movies like this masterpiece. Sentiment:
                        """)
               .options(ChatOptions.builder()
                     .model("gemma3:1b")
                     .temperature(0.1)
                     .maxTokens(5)
                     .build())
               .call()
               .content()
               .trim();
         System.out.println("Output: " + reviewSentiment);
      };
   }
}
```

Let's test this out; here's the output:

```
Started DemoApplication in 3.463 seconds (process running for 3.899)
Output: Negative
```

Building a RAG-Based Application

You will build upon the airlines chat application where users need information about the flight information like date of departure, from, to locations, departure/arrival airport, departure/arrival time, and current status. Since this data is available only to your airlines company, a foundational LLM would not have this information available to it. So, you will use the RAG architecture to retrieve this information from your internal source systems to provide information to your customers.

RAG essentially has three components to it: retrieval, augmentation, and generation. The following sections will focus on each of these.

Retrieval

The flight-related information could be in any data source including relational databases and file systems and in a variety of formats. You will need to read from this source all files relevant to you. This should ideally be a separate process where the system reads from the source new files every day on a periodic basis based on the business requirements. To keep it simple for this use case, you will read it when your Spring Boot application boots up for the first time.

You will store all of the flight information for your airlines company in a Markdown file named flights.md in the resources folder. The format of the Markdown file is similar to a CSV file where the first row contains field names like Aircraft, Data of Departure, From, etc., and the subsequent rows contain individual flights for the airline. Create a file at the following location and save the flight details. The information is also available in the source code available for this chapter.

File: ./src/main/resources/flights.md

```
Aircraft,Date of Departure,From (US City),To (US City),Departure Airport,Arrival Airport,Flight Number,Departure Time,Arrival Time,Status
Boeing 737,2023-11-01,New York,Los Angeles,JFK,LAX,AA101,08:00,11:00,On Time
Airbus A320,2023-11-01,Chicago,Miami,ORD,MIA,DL202,09:30,13:00,On Time
Boeing 787,2023-11-01,San Francisco,Seattle,SFO,SEA,UA303,10:15,12:00,Delayed
Airbus A321,2023-11-01,Houston,Dallas,IAH,DFW,SW404,11:45,13:15,On Time
Boeing 737,2023-11-01,Atlanta,Denver,ATL,DEN,AA505,12:30,15:00,On Time
Airbus A320,2023-11-01,Boston,Las Vegas,BOS,LAS,DL606,14:00,17:30,Cancelled
Boeing 787,2023-11-01,Philadelphia,Phoenix,PHL,PHX,UA707,15:45,18:30,On Time
Airbus A321,2023-11-01,Orlando,San Diego,MCO,SAN,SW808,16:30,19:00,On Time
Boeing 737,2023-11-02,New York,Chicago,JFK,ORD,AA909,07:00,09:00,On Time
Airbus A320,2023-11-02,Los Angeles,Houston,LAX,IAH,DL101,08:30,12:00,On Time
Boeing 787,2023-11-02,Miami,Atlanta,MIA,ATL,UA202,10:00,12:00,Delayed
Airbus A321,2023-11-02,Seattle,Dallas,SEA,DFW,SW303,11:15,14:30,On Time
Boeing 737,2023-11-02,Denver,Boston,DEN,BOS,AA404,13:00,17:00,On Time
Airbus A320,2023-11-02,Las Vegas,Philadelphia,LAS,PHL,DL505,14:30,18:00,Cancelled
Boeing 787,2023-11-02,Phoenix,Orlando,PHX,MCO,UA606,16:00,20:00,On Time
Airbus A321,2023-11-02,San Diego,New York,SAN,JFK,SW707,17:45,21:30,On Time
Boeing 737,2023-11-03,Chicago,Los Angeles,ORD,LAX,AA808,08:00,10:30,On Time
Airbus A320,2023-11-03,Houston,San Francisco,IAH,SFO,DL909,09:30,12:00,On Time
Boeing 787,2023-11-03,Atlanta,Seattle,ATL,SEA,UA101,11:00,14:00,Delayed
Airbus A321,2023-11-03,Dallas,Miami,DFW,MIA,SW202,12:15,15:30,On Time
Boeing 737,2023-11-03,Boston,Denver,BOS,DEN,AA303,13:30,16:30,On Time
Airbus A320,2023-11-03,Philadelphia,Las Vegas,PHL,LAS,DL404,15:00,18:30,Cancelled
Boeing 787,2023-11-03,Orlando,Phoenix,MCO,PHX,UA505,16:45,20:00,On Time
Airbus A321,2023-11-03,New York,San Diego,JFK,SAN,SW606,18:00,21:30,On Time
Boeing 737,2023-11-04,Los Angeles,Chicago,LAX,ORD,AA707,07:30,10:00,On Time
Airbus A320,2023-11-04,San Francisco,Houston,SFO,IAH,DL808,09:00,12:30,On Time
Boeing 787,2023-11-04,Seattle,Atlanta,SEA,ATL,UA909,10:30,13:30,Delayed
Airbus A321,2023-11-04,Miami,Dallas,MIA,DFW,SW101,11:45,14:45,On Time
Boeing 737,2023-11-04,Denver,Boston,DEN,BOS,AA202,13:00,17:00,On Time
Airbus A320,2023-11-04,Las Vegas,Philadelphia,LAS,PHL,DL303,14:30,18:00,Cancelled
Boeing 787,2023-11-04,Phoenix,Orlando,PHX,MCO,UA404,16:00,20:00,On Time
Airbus A321,2023-11-04,San Diego,New York,SAN,JFK,SW505,17:45,21:30,On Time
Boeing 737,2023-11-05,Chicago,Los Angeles,ORD,LAX,AA606,08:00,10:30,On Time
Airbus A320,2023-11-05,Houston,San Francisco,IAH,SFO,DL707,09:30,12:00,On Time
Boeing 787,2023-11-05,Atlanta,Seattle,ATL,SEA,UA808,11:00,14:00,Delayed
Airbus A321,2023-11-05,Dallas,Miami,DFW,MIA,SW909,12:15,15:30,On Time
Boeing 737,2023-11-07,Chicago,Los Angeles,ORD,LAX,AA404,08:00,10:30,On Time
```

```
Airbus A320,2023-11-07,Houston,San Francisco,IAH,SFO,DL505,09:30,12:00,On Time
Boeing 787,2023-11-07,Atlanta,Seattle,ATL,SEA,UA606,11:00,14:00,Delayed
Airbus A321,2023-11-07,Dallas,Miami,DFW,MIA,SW707,12:15,15:30,On Time
Boeing 737,2023-11-08,Los Angeles,Chicago,LAX,ORD,AA303,07:30,10:00,On Time
Airbus A320,2023-11-08,San Francisco,Houston,SFO,IAH,DL404,09:00,12:30,On Time
Boeing 787,2023-11-08,Seattle,Atlanta,SEA,ATL,UA505,10:30,13:30,Delayed
Airbus A321,2023-11-08,Miami,Dallas,MIA,DFW,SW606,11:45,14:45,On Time
```

As shown in Figure 8-8, the next step is to enable storage of all the available documents/data in your vector store. This will enable you to later retrieve or search from it based on a customer query. Vector stores can store the information in vector embeddings format. So, the goal is to read the files and divide them into small document chunks.

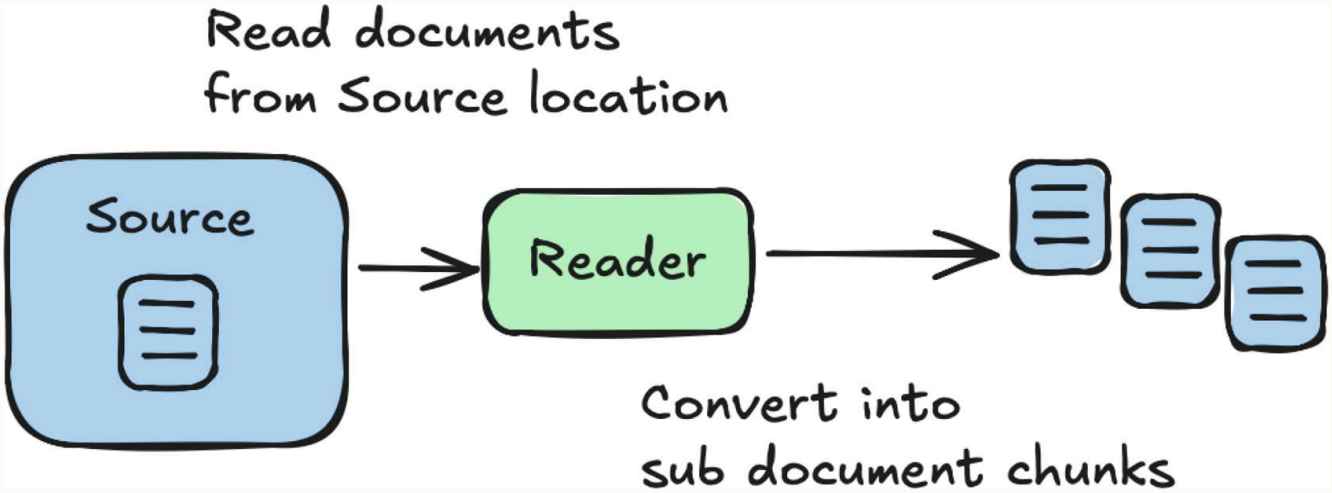


Figure 8-8 Read the source files and divide them into small chunks

The DocumentReaders API of Spring AI will help you to read the different source files. There are different readers for reading different files. The following is a list of available readers:

- JSON: The JsonReader reader processes JSON documents, converting them into a list of Document objects.
- Text: The TextReader reader processes plain text documents, converting them into a list of Document objects.
- HTML: The JsoupDocumentReader reader processes HTML documents, converting them into a list of Document objects using the JSoup library.
- Markdown: The MarkdownDocumentReader reader processes Markdown documents, converting them into a list of Document objects.
- PDF Page: The PagePdfDocumentReader reader uses Apache PdfBox library to parse PDF documents.
- Tika (DOCX, PPTX, HTML...): The TikaDocumentReader reader uses Apache Tika to extract text from a variety of document formats such as PDF, DOC/DOCX, PPT/PPTX, and HTML.

In this example, you will leverage the MarkdownDocumentReader to read your Markdown file. You will create a bean named documentReader that initializes the MarkdownDocumentReader to read the Markdown from the Spring Resource location. The resource has a default value for the flights.md file. MarkdownDocumentReader takes in two parameters. The first one is the resource location, and the second is the Markdown document reader config. The config states a few configuration values needed:

1. withHorizontalRuleCreateDocument: When set to true, horizontal rules in the Markdown will create new Document objects.
2. withIncludeCodeBlock: When set to true, code blocks will be included in the same Document object as the surrounding text. When false, code blocks create separate Document objects.
3. withIncludeBlockquote: When set to true, block quotes will be included in the same Document object as the surrounding text. When false, blockquotes create separate Document objects.

This step will enable you to have a bean available for the later steps to fetch documents read by the DocumentReader.

File: ./src/main/java/com/example/demo/DemoApplication.java

```
@Bean
public MarkdownDocumentReader documentReader(@Value("classpath:flights.md") Resource resource) {
   MarkdownDocumentReaderConfig config = MarkdownDocumentReaderConfig.builder()
         .withHorizontalRuleCreateDocument(true)
         .withIncludeCodeBlock(false)
         .withIncludeBlockquote(false)
         .withAdditionalMetadata("filename", "flights.md")
         .build();
  return new MarkdownDocumentReader(resource, config);
}
```

Now, you have the documents available to you in chunked format. The next step, as illustrated in Figure 8-9, is to convert each of these chunked documents from text to vector embeddings format. This can be done by passing them as inputs to any embedding model. You will use the OpenAI embedding model for this purpose.

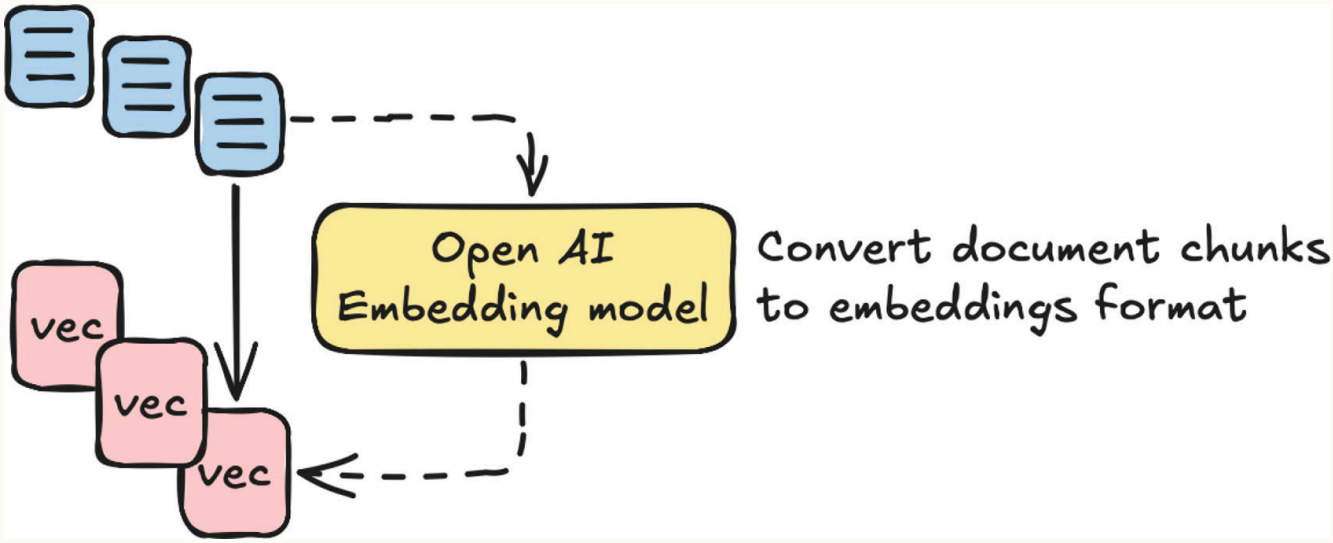


Figure 8-9 Convert chunked documents from text to vector embeddings

Once the documents are converted to vector embeddings, you need to store them in the vector store for later retrieval, as shown in Figure 8-10.

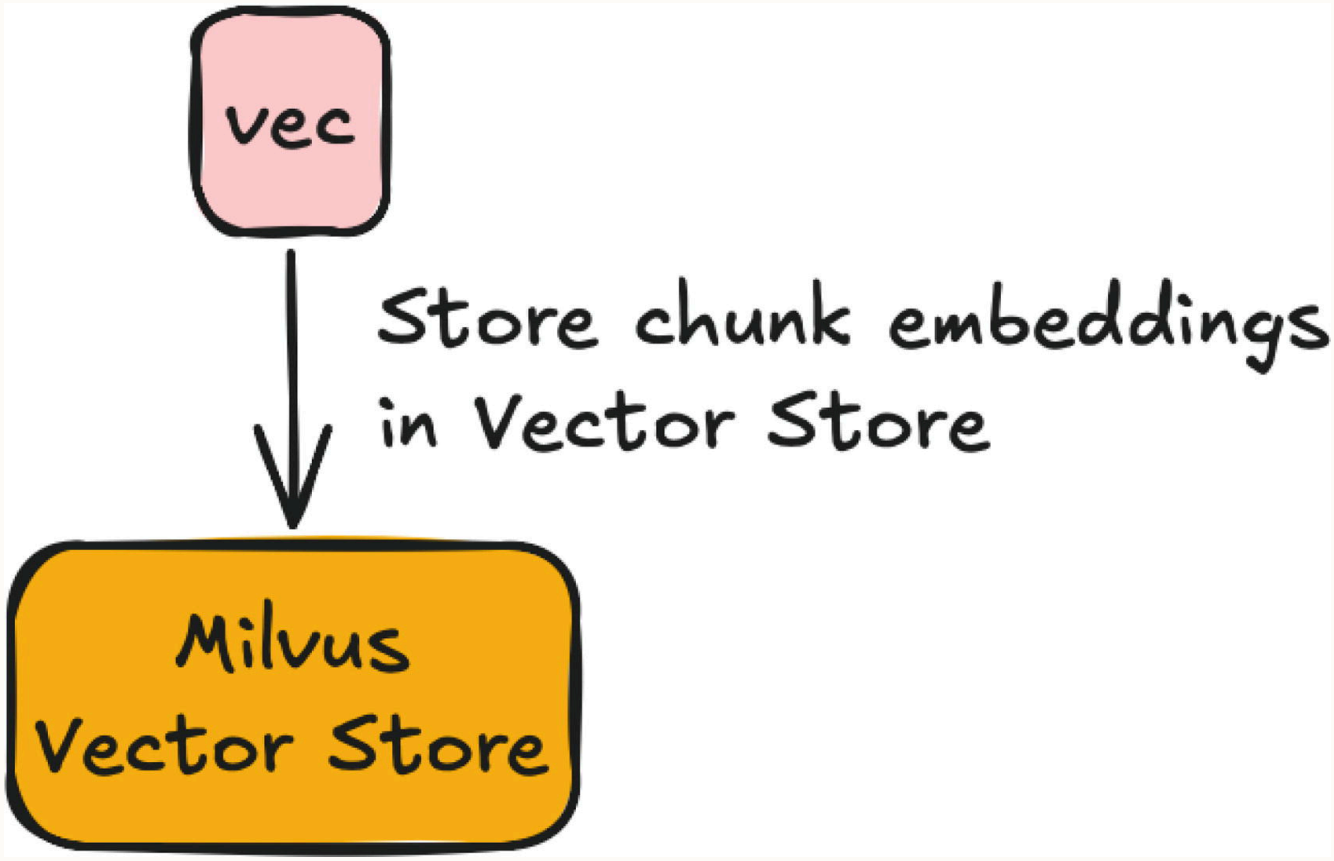


Figure 8-10 Storage of vector embeddings in vector store

Both the steps of converting the documents to embeddings and storing them in the vector store happen through the MilvusVectorStore API. Let's see how to achieve this. One of the prerequisites is to create a Milvus vector store. The easiest way is to do this through containers. So, make sure you have Docker/Podman installed in your system.

You will want to start a Milvus vector store container when your application starts so that you can use it to store the embeddings. This can be achieved by manually creating a container, but you will see the programmatic approach here for creating the container through a bean setup so that other parts of the code are able to access the container and have this dependency figured out.

Let's define a method named milvusContainer where you refer to the Docker Hub path for Milvusdb at milvusdb/milvus:latest and reference the container by creating a new instance of the MilvusContainer and passing the Docker hub name of the Milvus image. Then start the container using container.start() and return it for later reference. This enables the script to have a reference to the container and makes sure it is started before accessing it.

File: ./src/main/java/com/example/demo/DemoApplication.java

```
@Bean
public MilvusContainer milvusContainer() {
   MilvusContainer container = new MilvusContainer("milvusdb/milvus:latest");
   container.start();
   return container;
}
```

The MilvusServiceClient allows you to connect to the Milvus container created earlier. It takes in as argument the ConnectParam initialized with the container URI and database authorization credentials. Finally, you return the initialized client for later reference as a bean. This bean takes as argument the previously set up container and is injected at runtime, setting up the right dependency.

File: ./src/main/java/com/example/demo/DemoApplication.java

```
@Bean
public MilvusServiceClient milvusClient(MilvusContainer milvusContainer) {
   return new MilvusServiceClient(ConnectParam.newBuilder()
         .withAuthorization("minioadmin", "minioadmin")
         .withUri(milvusContainer.getEndpoint())
         .build());
}
```

The next step is to read the chunked documents, convert them to embeddings, and store them in the Milvus vector store. Let's create a bean named vecStore that returns the VectorStore for later reference. It will take as injected arguments the milvusClient and markDocumentReader objects created previously.

```
@Bean
public VectorStore vecStore(MilvusServiceClient milvusClient, MarkdownDocumentReader markdownDocumentReader) {
```

To create embeddings, you will need to access the OpenAI embedding model. You will create a reference to the OpenAI API through the use of the OpenAiApi API.

```
OpenAiApi openAiApi = new OpenAiApi(apiKey);
```

The openAiApi API can then be used to access the OpenAI embedding model through the OpenAiEmbeddingModel API. It takes as arguments the openAiApi reference, mode, and embedding model options.

```
OpenAiEmbeddingModel embeddingModel = new OpenAiEmbeddingModel(
      openAiApi,
      MetadataMode.EMBED,
      OpenAiEmbeddingOptions.builder()
            .model("text-embedding-ada-002")
            .user("user-6")
            .build(),
      RetryUtils.DEFAULT_RETRY_TEMPLATE);
```

Once this is set up, you can use the MilvusVectorStore API to set up vector store via the embeddingModel. It takes as arguments the milvusClient and embeddingModel. You also establish a few configurations like the collectionName and databaseName that you need to connect to along with the indexType and metricType for similarity comparison, which is COSINE in this case. The option for initializeSchema is true if it is not already present.

```
MilvusVectorStore vectorStore = MilvusVectorStore.builder(milvusClient, embeddingModel)
      .collectionName("vector_store")
      .databaseName("default")
      .indexType(IndexType.IVF_FLAT)
      .metricType(MetricType.COSINE)
      .batchingStrategy(new TokenCountBatchingStrategy())
      .initializeSchema(true)
      .build();
```

Once the vector store object is created, you will want to read the documents and add them to the store. The add method of vectorStore enables you to do so.

```
vectorStore.add(markdownDocumentReader.read());
```

The following is the code for the vecStore bean:

```
@Bean
public VectorStore vecStore(MilvusServiceClient milvusClient, MarkdownDocumentReader markdownDocumentReader) {
   OpenAiApi openAiApi = new OpenAiApi(apiKey);
   OpenAiEmbeddingModel embeddingModel = new OpenAiEmbeddingModel(
         openAiApi,
         MetadataMode.EMBED,
         OpenAiEmbeddingOptions.builder()
               .model("text-embedding-ada-002")
               .user("user-6")
               .build(),
         RetryUtils.DEFAULT_RETRY_TEMPLATE);
  MilvusVectorStore vectorStore = MilvusVectorStore.builder(milvusClient, embeddingModel)
         .collectionName("vector_store")
         .databaseName("default")
         .indexType(IndexType.IVF_FLAT)
         .metricType(MetricType.COSINE)
         .batchingStrategy(new TokenCountBatchingStrategy())
         .initializeSchema(true)
         .build();
  vectorStore.add(markdownDocumentReader.read());
   return vectorStore;
}
```

Figure 8-11 highlights the end-to-end flow of all the steps covered so far.

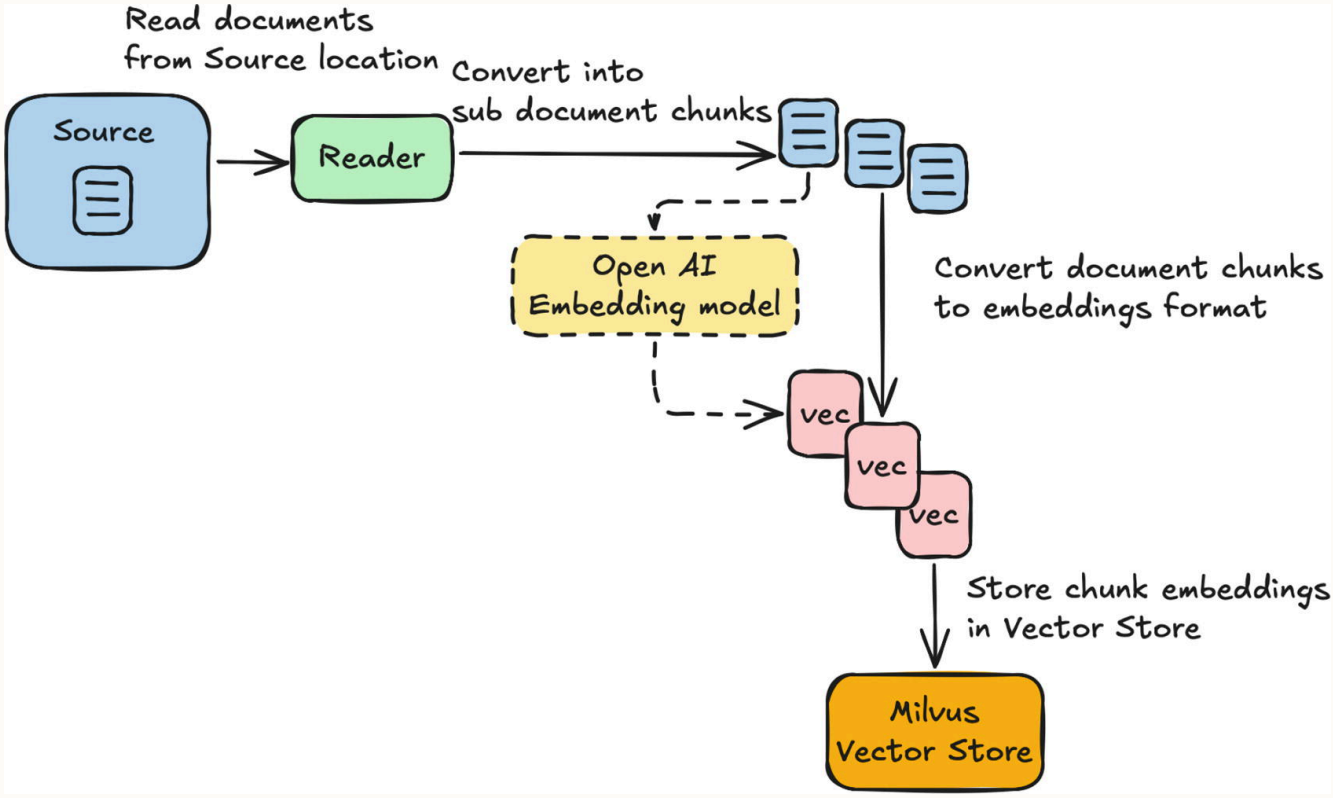


Figure 8-11 End-to-end flow of reading source documents, chunking them and storing them in the vector store in vector embeddings format

The code for the end-to-end flow shown in Figure 8-11 is as follows:

File: ./src/main/java/com/example/demo/DemoApplication.java

```
package com.example.demo;
import io.milvus.client.MilvusServiceClient;
import io.milvus.grpc.DataType;
import io.milvus.param.ConnectParam;
import io.milvus.param.IndexType;
```

```
import io.milvus.param.MetricType;
import io.milvus.param.collection.CreateCollectionParam;
import io.milvus.param.collection.FieldType;
import io.milvus.param.index.CreateIndexParam;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.document.Document;
import org.springframework.ai.document.MetadataMode;
import org.springframework.ai.embedding.TokenCountBatchingStrategy;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.ai.openai.OpenAiEmbeddingModel;
import org.springframework.ai.openai.OpenAiEmbeddingOptions;
import org.springframework.ai.openai.api.OpenAiApi;
import org.springframework.ai.reader.markdown.MarkdownDocumentReader;
import org.springframework.ai.reader.markdown.config.MarkdownDocumentReaderConfig;
import org.springframework.ai.retry.RetryUtils;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.ai.vectorstore.milvus.MilvusVectorStore;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.core.io.Resource;
import org.testcontainers.milvus.MilvusContainer;
import java.util.List;
import java.util.stream.Collectors;
@SpringBootApplication
public class DemoApplication {
  public static final int EMBEDDING_DIMENSION = 1536;
  @Value("${spring.ai.openai.api-key}"
   private String apiKey;
   public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
   @Bean
   public CommandLineRunner runner(ChatClient.Builder builder) {
      return args -> {
         ChatClient chatClient = builder.build();
         String response = chatClient.prompt("Tell me a joke").call().content();
         System.out.println(response);
      };
   }
   @Bean
   public OpenAiChatModel chatModel() {
      OpenAiApi openAiApi = new OpenAiApi(apiKey);
      return new OpenAiChatModel(openAiApi);
   }
   @Bean
   public VectorStore vecStore(MilvusServiceClient milvusClient, MarkdownDocumentReader markdownDocumentReader) {
      OpenAiApi openAiApi = new OpenAiApi(apiKey);
      OpenAiEmbeddingModel embeddingModel = new OpenAiEmbeddingModel(
            openAiApi,
            MetadataMode.EMBED,
            OpenAiEmbeddingOptions.builder()
                  .model("text-embedding-ada-002")
                  .user("user-6")
                  .build(),
            RetryUtils.DEFAULT_RETRY_TEMPLATE);
     MilvusVectorStore vectorStore = MilvusVectorStore.builder(milvusClient, embeddingModel)
```

```
            .collectionName("vector_store")
            .databaseName("default")
            .indexType(IndexType.IVF_FLAT)
            .metricType(MetricType.COSINE)
            .batchingStrategy(new TokenCountBatchingStrategy())
            .initializeSchema(true)
            .build();
        vectorStore.add(markdownDocumentReader.read());
        return vectorStore;
    }
    @Bean
    public MilvusContainer milvusContainer() {
        MilvusContainer container = new MilvusContainer("milvusdb/milvus:latest");
        container.start();
        return container;
    }
    @Bean
    public MilvusServiceClient milvusClient(MilvusContainer milvusContainer) {
        return new MilvusServiceClient(ConnectParam.newBuilder()
            .withAuthorization("minioadmin", "minioadmin")
            .withUri(milvusContainer.getEndpoint())
            .build());
    }
    @Bean
    public MarkdownDocumentReader documentReader(@Value("classpath:flights.md") Resource resource) {
        MarkdownDocumentReaderConfig config = MarkdownDocumentReaderConfig.builder()
            .withHorizontalRuleCreateDocument(true)
            .withIncludeCodeBlock(false)
            .withIncludeBlockquote(false)
            .withAdditionalMetadata("filename", "flights.md")
            .build();
        return new MarkdownDocumentReader(resource, config);
    }
}
```

With this foundational setup done, you can stitch together the retrieval, augmentation, and generation steps. As shown in Figure 8-12, to retrieve documents similar to the search query of the customer, you have to first convert it to vector embeddings.

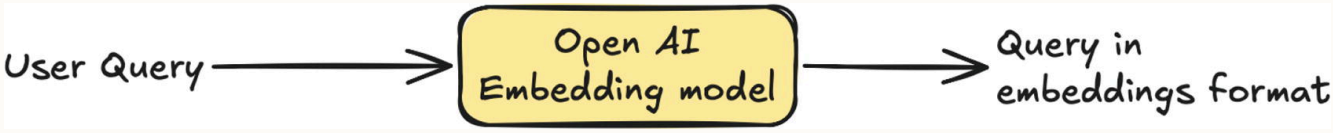


Figure 8-12 Convert the user query to vector embeddings using the OpenAI embedding model

Once you have the customer query in embeddings format, you can pass it to the vector store to do a vector similarity using COSINE similarity to retrieve the matched documents, as shown in Figure 8-13.

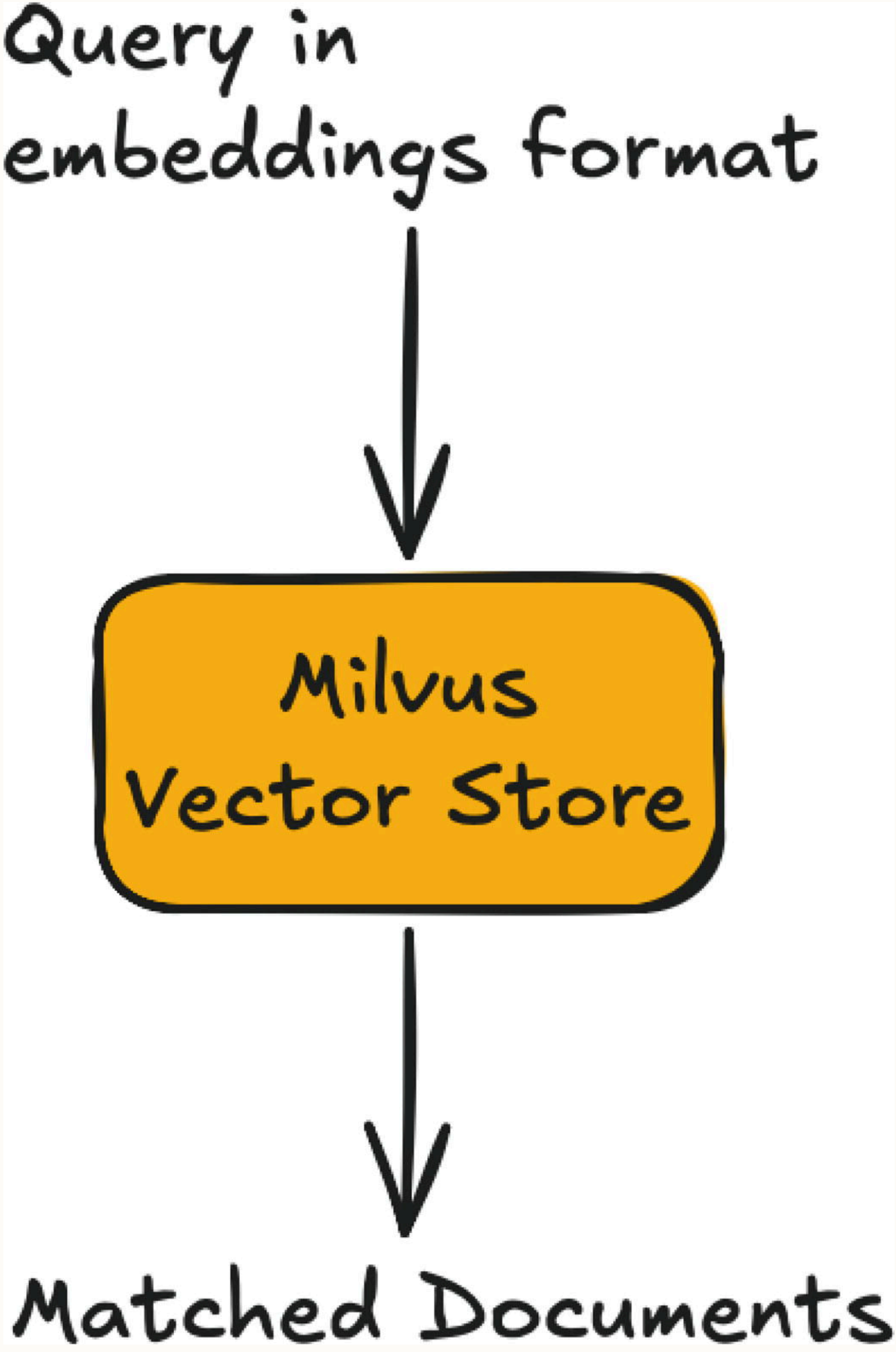


Figure 8-13 Retrieve the matched documents based on an embeddings similarity match

The matched documents along with the customer query are then passed to the foundational LLM model to create a human-friendly response in the context of the matched documents, as depicted in Figure 8-14.

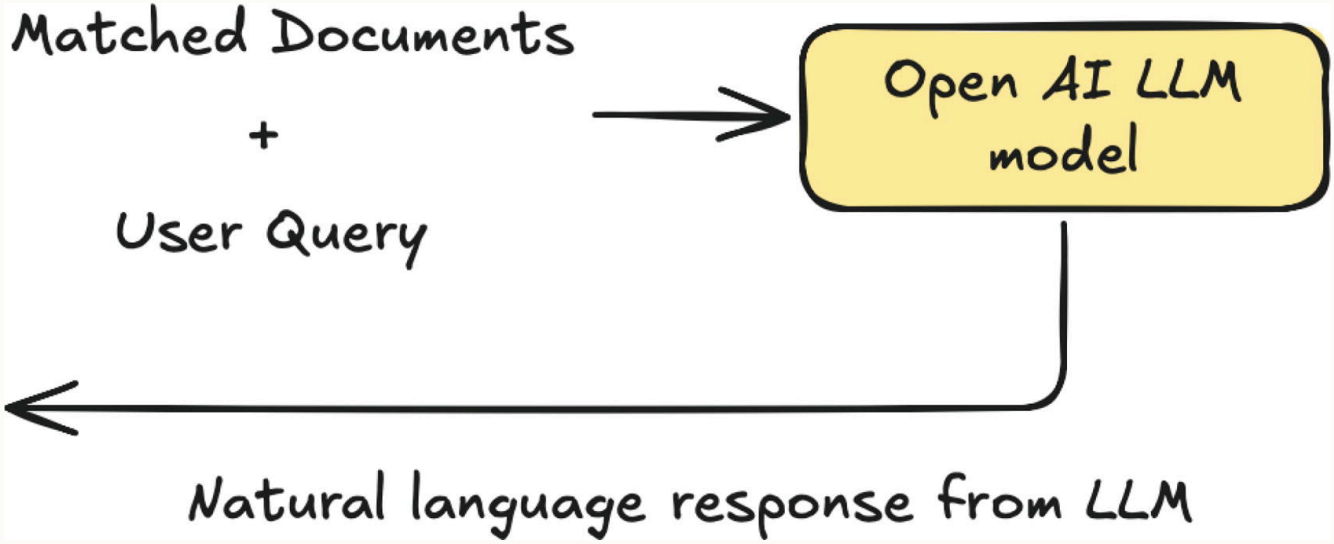


Figure 8-14 Generate a response from the foundation LLM

Figure 8-15 shows all the steps.

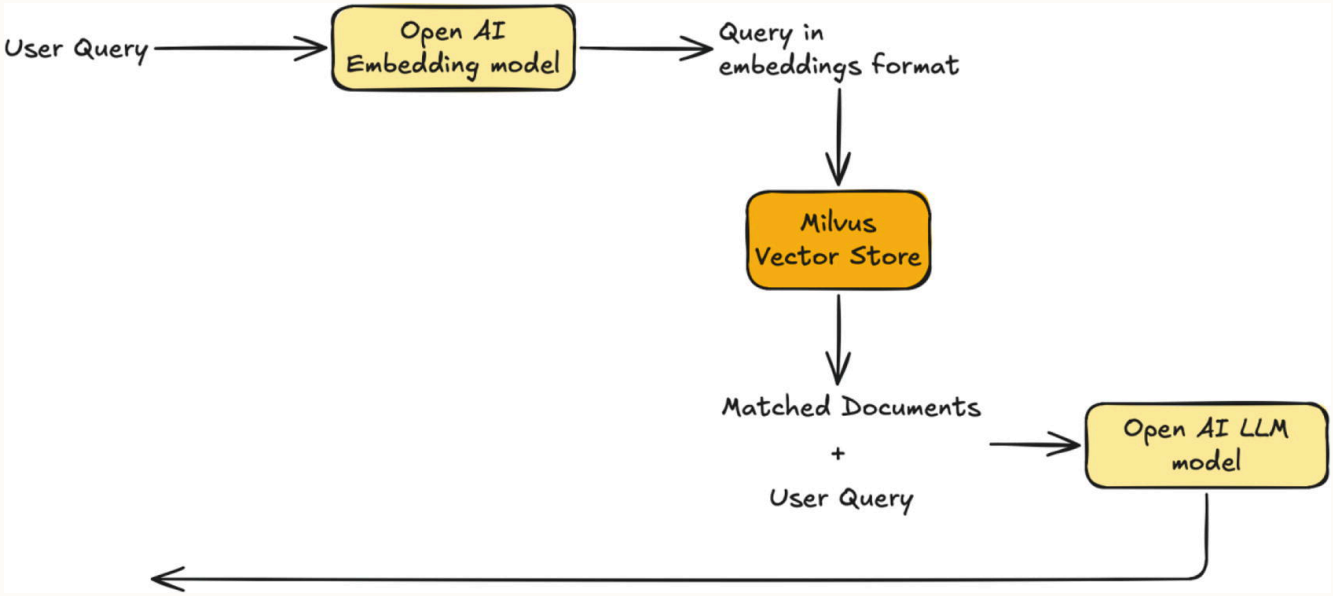


Figure 8-15 RAG Flow

Let's implement this. We will create a ChatService class to create a service layer abstraction that can be called from the Controller layer. It needs two arguments to be injected at runtime, namely, chatModel and vectorStore.

```
@Autowired
public ChatService(OpenAiChatModel chatModel, VectorStore vectorStore) {
   this.chatModel = chatModel;
   this.vectorStore = vectorStore;
}
```

The ChatClient API provides a helpful feature to enable the RAG model. Its builder interface takes in as an argument the chatModel along with the vectorStore and userText input to convert the query to embeddings, search on the vectorStore, and get the final response from the foundational LLM. The following is the code to achieve this:

```
public String answerQuestion(String userText) {
   ChatResponse chatResponse = ChatClient.builder(chatModel)
         .build()
         .prompt()
         .advisors(new QuestionAnswerAdvisor(vectorStore))
         .user(userText)
         .call()
```

```
            .chatResponse();
        return chatResponse.getResult().getOutput().getText().trim();
    }
```

The whole code for the ChatService file is shown here:

File: ./src/main/java/com/example/demo/service/ChatService.java

```
package com.example.demo.service;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.ai.openai.OpenAiChatModel;
import org.springframework.ai.openai.api.OpenAiApi;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class ChatService {
  private final OpenAiChatModel chatModel;
   private final VectorStore vectorStore;
  @Autowired
   public ChatService(OpenAiChatModel chatModel, VectorStore vectorStore) {
      this.chatModel = chatModel;
      this.vectorStore = vectorStore;
   }
  public String answerQuestion(String userText) {
      ChatResponse chatResponse = ChatClient.builder(chatModel)
            .build()
            .prompt()
            .advisors(new QuestionAnswerAdvisor(vectorStore))
            .user(userText)
            .call()
            .chatResponse();
      return chatResponse.getResult().getOutput().getText().trim();
   }
}
```

The next step is to collect user input through the controller and request a response using the Service class. You will create a ChatRequest class to infer user request arguments in the ChatController class.

File: ./src/main/java/com/example/demo/controller/ChatRequest.java

```
package com.example.demo.controller;
public class ChatRequest {
  private String userText;
  public ChatRequest() {}
  public ChatRequest(String userText) {
      this.userText = userText;
   }
  public String getUserText() {
      return userText;
   }
  public void setUserText(String userText) {
      this.userText = userText;
   }
}
```

File: ./src/main/java/com/example/demo/controller/ChatController.java

```
package com.example.demo.controller;
import com.example.demo.service.ChatService;
import org.springframework.ai.chat.model.ChatResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
@RequestMapping("/api/chat")
public class ChatController {
  private final ChatService chatService;
  @Autowired
   public ChatController(ChatService chatService) {
      this.chatService = chatService;
   }
  @PostMapping
   public String postChat(@RequestBody ChatRequest request) {
      return chatService.answerQuestion(request.getUserText());
   }
}
```

With all the components set up, you can now test the application by booting it up and running the following curl command from another terminal interface:

```
curl -X POST -H "Content-Type: application/json" -d '{"userText": "What is the flight details for DL606?"}' http://localhost:8080/api/chat
```

Output:

```
The flight details for DL606 are as follows:
- Date of Departure: 2023-11-01
- From: Boston
- To: Las Vegas
- Departure Airport: BOS
- Arrival Airport: LAS
- Departure Time: 14:00
- Arrival Time: 17:30
- Status: Cancelled
```

Wow, this is some great output from the API; it contains detailed information from the contextual data that the airlines company had.

Conclusion

In this chapter, you explored how to seamlessly integrate the Spring AI library with Spring Boot to build AI-driven applications. You started by understanding the purpose and features of Spring AI, reviewing its flexibility, portability, and ease of integration with various AI model providers and vector databases. You learned how Spring AI simplifies complex integrations through intuitive APIs like ChatClient, allowing easy communication with AI models through prompts, system, and user messages.

You then saw how to build a basic AI-powered application, initially using the ChatClient API to interact with OpenAI's models directly. Further, you extended our application to implement RAG by integrating the Milvus vector database,

showcasing the capability to handle advanced, context-aware queries.

Through these examples, it is clear that Spring AI significantly reduces complexity, providing a familiar, developer-friendly framework that aligns well with the Spring ecosystem, enabling efficient development of sophisticated AI-powered Spring Boot applications. The next chapter will cover prompt engineering.

Previous chapter
< 7. From Chatty to Clever: Retrieval-Augmented Generation

Next chapter
9. Prompt Alchemy: Patterns That Make Models Look Smarter >